

Kali & OSINT Tools — Cheat Sheet

The essential command-line tools for open-source investigation: what each one does and how to run it.

Authorisation first. Recon, content discovery, and hash cracking are powerful and, on systems you do not own, often illegal. Use these only on your own assets, on the training targets, or within a written engagement scope. Throttle, respect Terms of Service, and record provenance (URL, UTC, hash).

1. Username enumeration — where does a handle exist?

sherlock — checks a username across ~400 sites.

```
sherlock johndoe          # search one handle
sherlock johndoe --print-found  # only show hits
sherlock user1 user2 --no-txt   # several handles, no output file
sherlock johndoe --proxy socks5://127.0.0.1:9050 # via Tor
```

Treat results as **leads**: some sites return false positives. Open the URL to confirm.

maigret — like Sherlock, more sites, richer report.

```
maigret johndoe          # console report
maigret johndoe --html    # HTML report with details
maigret johndoe -a        # all sites, including slow ones
```

whatsmyname (web + [whatsmyname](#) CLI) — curated, low-false-positive list. Also usable via the site [whatsmyname.app](#).

2. Email footprint

holehe — which services have an account for this email (no email sent to the target for most modules).

```
holehe target@example.com # list services
holehe target@example.com --only-used # hide "not used" lines
```

Have I Been Pwned — breach exposure (via the site, or API with a key).

```
curl -s -H "hibp-api-key: $HIBP_KEY" \
  "https://haveibeenpwned.com/api/v3/breachedaccount/target@example.com"
```

Pwned Passwords — is a password in a breach, without sending it (k-anonymity):

```
# send only the first 5 chars of the SHA-1, match the rest locally
pass="Winter2024!"; h=$(printf '%s' "$pass" | openssl sha1 | awk '{print toupper($2)}')
curl -s "https://api.pwnedpasswords.com/range/${h:0:5}" | grep -i "${h:5}"
```

3. curl — the Swiss army knife for HTTP & APIs

```
curl -s URL          # fetch silently
curl -sL URL         # follow redirects
curl -I URL          # headers only (HEAD)
curl -A "Mozilla/5.0" URL # set User-Agent
curl -H "Authorization: Bearer $T" URL # add a header (API auth)
curl -H "Accept: application/json" URL # ask for JSON
curl -d 'a=1&b=2' URL # POST form data
curl -X POST -d '{"k":"v"}' -H "Content-Type: application/json" URL
curl -o file.jpg URL # save to a file
curl --socks5-hostname 127.0.0.1:9050 URL # route through Tor
curl -s URL | jq '.' # pretty-print / query JSON
```

Certificate Transparency → **subdomains (crt.sh)**:

```
curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '[][.name_value]' | sort -u
```

Wayback Machine → historical URLs:

```
curl -s "http://web.archive.org/cdx/search/cdx?url=example.com*&output=text&fl=original&collapse=urlkey"
```

4. jq — parse and query JSON

```
curl -s API | jq '.' # pretty-print
jq '.items[] | .name' file.json # extract a field from an array
jq -r '[][.email]' file.json # -r = raw (no quotes)
jq '.data | length' file.json # count elements
```

5. Encoding & decoding

base64:

```
printf 'OSINT' | base64 # encode -> T1NJTLQ=
printf 'T1NJTLQ=' | base64 -d # decode -> OSINT
```

hex:

```
printf 'OSINT' | xxd # hex + ASCII dump
printf 'OSINT' | xxd -p # plain hex
echo '4f53494e54' | xxd -r -p # hex -> text
```

ROT13 / case / URL:

```
echo "hello" | tr 'A-Za-z' 'N-ZA-Mn-za-m' # ROT13
python3 -c "import urllib.parse; print(urllib.parse.unquote(sys.argv[1]))" "a%20b"
```

For anything more exotic (multi-layer, XOR, magic detection), use **CyberChef** (gchq.github.io/CyberChef).

6. Hashes — identify then crack (authorised only)

Identify a hash:

```
hashid '5f4dcc3b5aa765d61d8327deb882cf99' # guesses the algorithm
hash-identifier # interactive
```

hashcat — GPU cracker. **-m** = hash mode, **-a** = attack mode.

```
hashcat -m 0 -a 0 hashes.txt rockyou.txt # MD5, dictionary
hashcat -m 100 -a 0 hashes.txt rockyou.txt # SHA1, dictionary
hashcat -m 1000 -a 0 hashes.txt rockyou.txt # NTLM
hashcat -m 0 -a 3 hashes.txt '?d?d?d?d?d?' # MD5, 6-digit mask (brute)
hashcat -m 22000 -a 0 wifi.hc22000 rockyou.txt # WPA/WPA2
hashcat --show hashes.txt -m 0 # show already-cracked
```

Masks: **?d** digit, **?l** lower, **?u** upper, **?s** symbol, **?a** any.

john the ripper — CPU cracker, great auto-detection.

```
john --wordlist=rockyou.txt hashes.txt
john --format=raw-md5 hashes.txt
john --show hashes.txt # results
```

Wordlist on Kali: </usr/share/wordlists/rockyou.txt> ([gunzip](#) it first).

7. Web content discovery (directories & files)

gobuster — fast, modern.

```
gobuster dir -u https://target -w /usr/share/wordlists/dirb/common.txt
gobuster dir -u https://target -w list.txt -x php,txt,bak # try extensions
gobuster dns -d target.com -w subdomains.txt # subdomains
```

ffuf — fuzzing, **FUZZ** marks the injection point.

```
ffuf -u https://target/FUZZ -w list.txt
ffuf -u https://target/FUZZ -w list.txt -mc 200,301 -fc 404
ffuf -u "https://target/?q=FUZZ" -w params.txt # fuzz a parameter
```

dirb / dirbuster — the classic (dirbuster is the GUI, dirb the CLI).

```
dirb https://target /usr/share/wordlists/dirb/common.txt
```

whatweb / wappalyzer — fingerprint the tech stack.

```
whatweb https://target
```

8. DNS & domain recon

```
whois example.com # registration info
dig example.com +short # A record, terse
dig MX example.com +short # mail servers
dig -x 8.8.8.8 +short # reverse (IP -> name)
dig AXFR example.com @ns1.example.com # zone transfer (if misconfigured)
host example.com # quick lookup
```

Subdomain enumeration:

```
subfinder -d example.com -silent # passive subdomains
amass enum -passive -d example.com # broader passive recon
subfinder -d example.com -silent | httpx -silent # which are alive
```

9. Ports & services — nmap

```
nmap target # top 1000 ports
nmap -p- target # all 65535 ports
nmap -sV target # detect service versions
nmap -sV -sC target # versions + default scripts
nmap -Pn target # skip ping (host blocks ICMP)
nmap -sV -oA scan target # save in all formats
```

Scanning hosts you do not own can be illegal. Stay in scope.

10. Metadata & files

exiftool — read/strip metadata (photos, PDFs, docs).

```
exiftool photo.jpg # all metadata
exiftool -GPSPosition -DateTimeOriginal -Model photo.jpg
exiftool -all= -overwrite_original photo.jpg # STRIP all metadata
exiftool -csv -r dir/ > meta.csv # bulk export to CSV
```

Other quick ones: **file suspect** (what is this file?), **strings binary | less** (readable text inside a file), **binwalk file** (find embedded files), **steghide extract -sf img.jpg** (steganography).

11. Aggregate recon frameworks

```
theHarvester -d example.com -b all      # emails, subdomains, hosts from many sources
spiderfoot -l 127.0.0.1:5001           # web UI, automated OSINT graph
recon-ng                               # modular recon console (marketplace of modules)
```

12. Staying anonymous while you work

Tor + proxychains — route any CLI tool through Tor.

```
sudo service tor start                  # start Tor (SOCKS on 9050)
proxychains4 curl https://ifconfig.me   # curl via Tor
proxychains4 sherlock johndoe          # torify a tool that has no proxy flag
torsocks nmap -sT target                # torify with torsocks (TCP connect only)
```

Check your exit:

```
curl --socks5-hostname 127.0.0.1:9050 https://check.torproject.org/api/ip
```

13. Google / search dorking (no tool needed)

Operator	Finds
<code>site:example.com</code>	pages on one domain
<code>filetype:pdf</code>	a file type
<code>intitle:"index of"</code>	open directory listings
<code>inurl:admin</code>	keyword in the URL
<code>"exact phrase"</code>	exact match
<code>-word</code>	exclude a word
<code>cache:URL</code>	Google's cached copy

Combine: `site:example.com filetype:xlsx confidential`.

Where to point these tools in this course

- Handle/email pivots → sherlock, maigret, holehe (Day 1 SecOps lab, Who Am I lab).
- Recursive email harvesting → curl + regex (Day 2 Email Harvester lab).
- API scraping → curl + Burp + Python (Day 2 API Scraping lab).
- Metadata & forensics → exiftool, file, strings (Day 2 Media Forensics lab).
- Subdomains/infra pivots → crt.sh, subfinder, nmap.

Keep a lab notebook: for every command, record the target, the time (UTC), and the result. Reproducibility and provenance are what separate an investigation from a guess.